Smart MCQ Solver: A Comparative Study of Classical Machine Learning, Transformer Models, Retrieval-Augmented Generation, and Scratch Deep Learning Architectures

Course: Deep Learning and Generative AI

Student: Aryan Bose

Roll Number: 24F3004686

Institute: Indian Institute of Technology Madras

Abstract

Multiple Choice Question Answering (MCQA) is a challenging Natural Language Processing (NLP) task that requires selecting the most appropriate answer from a set of candidate options. This project presents the development of a complete MCQA system through a progressive exploration of multiple machine learning paradigms. Beginning with classical machine learning techniques, the work advances through transformer-based language models, retrieval-augmented generation (RAG), and finally a deep learning model developed entirely from scratch.

Several approaches were implemented and evaluated, including TF-IDF with Logistic Regression, Feed Forward Neural Networks, transformer architectures such as BERT, SciBERT, DistilRoBERTa, ConvBERT, and ELECTRA, a retrieval pipeline using Sentence Transformers, FAISS, Cross-Encoder reranking, and zero-shot classification, and a Bidirectional Long Short-Term Memory (BiLSTM) network with an attention mechanism. The comparative analysis demonstrates the strengths and limitations of each approach while highlighting the importance of representation learning, retrieval quality, and model architecture in solving MCQA tasks.

The project provides an end-to-end machine learning pipeline encompassing data preprocessing, exploratory data analysis, model development, evaluation, and comparative performance analysis, offering a comprehensive study of modern NLP techniques for multiple-choice question answering.

1. Introduction

Multiple Choice Question Answering (MCQA) is an important Natural Language Processing task in which a system must identify the correct answer from a fixed set of candidate options. MCQA systems have applications in educational technology, intelligent tutoring systems, online examinations, and automated assessment platforms.

The Smart MCQ Solver project investigates multiple approaches for solving this task, ranging from traditional machine learning algorithms to modern transformer-based language models and retrieval-augmented methods. In addition to evaluating pretrained models, a complete deep learning architecture was implemented from scratch to better understand the underlying learning process.

The primary objective of this project is not only to maximize predictive performance but also to compare different paradigms for solving MCQA problems. Through systematic experimentation, the project evaluates how classical feature engineering, contextual language representations, retrieval mechanisms, and custom neural architectures influence model performance and generalization

2. Problem Statement

The objective of the Smart MCQ Solver project is to develop an intelligent system capable of answering multiple-choice questions by selecting the most appropriate answer from five candidate options (A–E). Unlike traditional text classification tasks, the model must understand the semantic relationship between the question and each candidate answer before making a prediction.

Rather than relying on a single modeling approach, this project explores and compares multiple Natural Language Processing techniques, including classical machine learning algorithms, transformer-based language models, retrieval-augmented generation (RAG), and a deep learning model implemented entirely from scratch. The goal is to evaluate the effectiveness of each approach and identify the most suitable solution for the MCQA task.

3. Dataset Description

The Smart MCQ Solver dataset consists of multiple-choice questions where each training sample contains a question prompt, five candidate answer options, and the corresponding correct answer. The test dataset follows the same structure but does not include the answer labels.

Column	Description
id	Unique question identifier
prompt	Question statement
A – E	Candidate answer options
answer	Correct answer label (training only)

To prepare the data for training, the question prompt and all five answer options were combined into a single textual sequence. This representation enables the models to learn contextual relationships between the question and its candidate answers while maintaining a consistent input format across different experiments.

4. Evaluation Metric

Model performance was evaluated using Mean Average Precision at 3 (MAP@3), the official evaluation metric of the Smart MCQ Solver Challenge. Instead of predicting only a single answer, each model generates a ranked list of candidate answers. A prediction receives a higher score when the correct answer appears closer to the top of the ranked list.

In addition to MAP@3, validation accuracy and training loss were monitored during model development to assess convergence and generalization performance.

5. Overall Project Workflow

The project was developed through a sequence of progressively advanced approaches, beginning with traditional machine learning techniques and culminating in retrieval-augmented generation and a custom deep learning architecture.

[Smart MCQ Solver]

→

[EDA]

→

[Preprocessing]

→

[TF-IDF + LR]

→

[Transformer Models]

→

[RAG Pipeline]

→

[Scratch BiLSTM]

→

[Results & Comparison]

Summary

The Smart MCQ Solver project investigates multiple paradigms for solving multiple-choice question answering, beginning with classical machine learning and progressing toward modern transformer architectures, retrieval-augmented generation, and a custom deep learning model. This progressive methodology enables a comprehensive comparison of different approaches while highlighting their respective strengths and limitations.

6. Methodology

To investigate different approaches for solving the Smart MCQ Solver challenge, a series of experiments were conducted using progressively more advanced Natural Language Processing techniques. Each approach was evaluated under a consistent experimental framework to compare predictive performance and computational efficiency.

6.1 Classical Machine Learning

The initial baseline was developed using Term Frequency–Inverse Document Frequency (TF-IDF) feature extraction combined with Logistic Regression. For each question, the prompt was concatenated with each candidate answer option to create prompt–option pairs. These textual inputs were transformed into sparse numerical vectors using unigram, bigram, and trigram representations before classification.

Although computationally efficient, this approach relied solely on lexical similarity and could not effectively capture deeper semantic relationships between the question and answer choices.

6.2 Feed Forward Neural Network

A simple Feed Forward Neural Network (FFNN) was implemented as an initial deep learning baseline. Dense textual representations were provided as input to multiple fully connected layers, enabling the model to learn nonlinear decision boundaries.

While the FFNN demonstrated improved representation learning compared to classical machine learning techniques, its inability to model sequential dependencies limited its overall performance on complex multiple-choice questions.

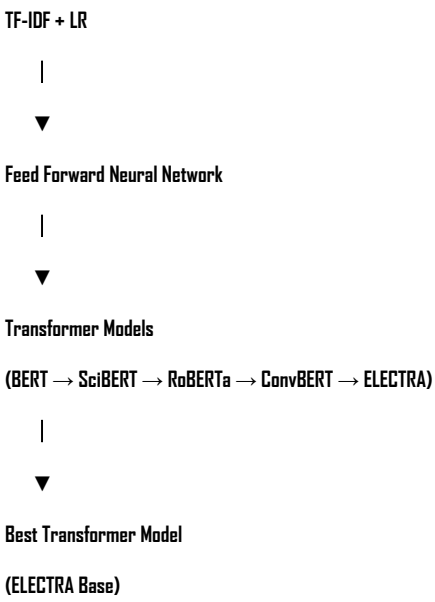
6.3 Transformer-based Language Models

Several pretrained transformer architectures were fine-tuned to leverage contextual language representations learned from large-scale corpora. Each model received a formatted input sequence containing the question followed by all five candidate answer options.

The following transformer models were evaluated:

- BERT Base
- SciBERT
- RoBERTa
- DistilRoBERTa
- XLM-RoBERTa
- ConvBERT
- ELECTRA Base
- ELECTRA Large

All transformer models were trained using a common preprocessing pipeline and comparable hyperparameter settings to ensure a fair comparison. Among the evaluated architectures, ELECTRA Base consistently achieved the strongest performance and was selected as the best transformer model for the project.



Summary

The methodology progressed from traditional feature-engineering approaches to contextual transformer architectures, allowing a systematic comparison of different modeling paradigms. This progression established a strong baseline before introducing retrieval-augmented techniques and a custom deep learning model in the subsequent sections.

7. Retrieval-Augmented Generation (RAG)

To investigate whether external contextual information could improve multiple-choice question answering, a Retrieval-Augmented Generation (RAG) pipeline was implemented. Instead of predicting answers solely from the question, the pipeline first retrieves relevant knowledge from a document corpus and incorporates it into the prediction process.

The retrieval process consists of four stages: document embedding, similarity search, reranking, and final answer prediction.

7.1 Dense Retrieval using Sentence Transformers

Each document in the knowledge base was converted into a dense vector representation using the all-MiniLM-L6-v2 Sentence Transformer. The same embedding model was used to encode the input question, enabling semantic similarity search within a shared embedding space.

Dense embeddings allow retrieval based on semantic meaning rather than exact keyword matching.

7.2 FAISS-based Similarity Search

The document embeddings were indexed using Facebook AI Similarity Search (FAISS). Given an input question, the corresponding embedding was queried against the FAISS index to retrieve the Top-5 most relevant documents based on vector similarity.

This retrieval stage provides candidate contexts that may contain information useful for answering the question.

7.3 Cross-Encoder Reranking

Although FAISS efficiently retrieves semantically similar documents, the retrieved ranking is not always optimal. To improve retrieval quality, the cross-encoder/ms-marco-MiniLM-L-6-v2 model was used to score each retrieved document jointly with the input question.

The document receiving the highest cross-encoder relevance score was selected as the final context for downstream prediction.

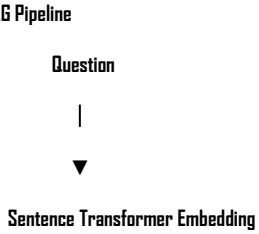
7.4 Zero-Shot Answer Prediction

The selected context and the original question were combined to form an augmented input in the format:

Context: [Retrieved Document]

Question: [Input Prompt]

This augmented input was passed to the facebook/bart-large-mnli zero-shot classification model. The five answer options (A–E) were supplied as candidate labels, and the model generated confidence scores for each option. The options were then ranked according to their predicted probabilities to produce the final answer.





Summary

The RAG framework combines semantic retrieval with neural reranking to provide additional contextual knowledge before prediction. Compared to relying solely on the question text, this approach improves information retrieval quality and enables the classifier to make more informed decisions by incorporating relevant external context.

8. Scratch Deep Learning Model

To better understand the fundamentals of deep learning for Natural Language Processing, a custom neural network was implemented entirely from scratch using PyTorch. Unlike transformer-based approaches, this model does not utilize any pretrained embeddings or language models. Instead, all parameters were initialized randomly and learned directly from the Smart MCQ Solver dataset during training.

8.1 Data Preparation

Prior to training, the dataset underwent several preprocessing steps to transform the raw text into a format suitable for neural network training.

The preprocessing pipeline included:

- Text cleaning and normalization

- Concatenation of the question and answer options
- Vocabulary construction from the training corpus
- Tokenization into integer indices
- Sequence padding to a fixed maximum length
- Label encoding of answer classes (A–E)

The processed data was then converted into PyTorch datasets and loaded using mini-batch DataLoaders for efficient training.

8.2 Model Architecture

The proposed scratch model consists of four primary components:

- Embedding Layer: Learns dense vector representations of words directly from the training data.
- Bidirectional LSTM: Processes text sequences in both forward and backward directions to capture contextual dependencies.
- Attention Mechanism: Assigns higher importance to informative words within the sequence, enabling the model to focus on the most relevant information.
- Fully Connected Classifier: Produces the final probability distribution across the five answer classes.



8.3 Training Strategy

The model was trained using a supervised learning framework with a stratified train-validation split. Cross Entropy Loss was employed as the optimization objective, while the Adam optimizer was used for parameter updates. Validation performance was monitored after each epoch, and the model checkpoint with the highest validation accuracy was retained for inference.

Training Configuration

Parameter	Value
Embedding Dimension	128
Hidden Dimension	128
LSTM Layers	2
Batch Size	32
Learning Rate	0.001
Optimizer	Adam
Loss Function	Cross Entropy Loss
Dropout	0.3
Epochs	10

Summary

The scratch BiLSTM with Attention model demonstrates that competitive multiple-choice question answering systems can be developed without pretrained language models. Although transformer architectures achieved superior leaderboard performance, the custom implementation provided valuable insights into sequence modeling, attention mechanisms, and end-to-end neural network training while establishing a strong baseline for future improvements.

9. Experimental Results and Comparative Analysis

A series of experiments were conducted to evaluate different approaches for solving the Smart MCQ Solver challenge. Beginning with classical machine learning techniques, progressively more advanced models—including transformer architectures, retrieval-augmented generation, and a custom deep learning model—were implemented and analyzed.

The performance of transformer-based models was evaluated using the Kaggle leaderboard score (MAP@3), while the scratch BiLSTM model was assessed using validation metrics due to its different experimental objective.

9.1 Model Performance Comparison

Approach	Model	Performance
Classical Machine Learning	TF-IDF + Logistic Regression	0.75436
Neural Network	Feed Forward Network	0.73482
Transformer	RoBERTa	0.74646
Transformer	XLM-RoBERTa	0.74729

Approach	Model	Performance
Transformer	ConvBERT	0.75353
Transformer	DistilRoBERTa	0.75561
Transformer	ELECTRA Base	0.76059
Transformer	ELECTRA Large	0.75519
Retrieval-Augmented Generation	FAISS + Cross Encoder + BART	Experimental Evaluation
Scratch Deep Learning	BiLSTM + Attention	Validation Performance

Note: The RAG pipeline and scratch BiLSTM were implemented to investigate alternative approaches and architectural understanding rather than to compete directly on the Kaggle leaderboard.

9.2 Discussion

The experimental results highlight the progression from traditional machine learning to advanced Natural Language Processing techniques.

The TF-IDF with Logistic Regression baseline achieved competitive performance despite its simplicity, demonstrating that lexical features remain effective for multiple-choice question answering. However, its reliance on sparse representations limited its ability to capture contextual semantics.

Transformer-based architectures consistently improved predictive performance by leveraging contextual language representations. Among the evaluated models, ELECTRA Base achieved the highest Kaggle leaderboard score (0.76059), providing the best balance between model capacity, computational efficiency, and generalization.

The Retrieval-Augmented Generation (RAG) pipeline demonstrated the effectiveness of combining semantic retrieval with neural reranking. By integrating FAISS retrieval, Cross-Encoder reranking, and zero-shot classification, the system was able to incorporate external contextual information before prediction, offering an alternative inference strategy beyond conventional fine-tuning.

The scratch BiLSTM with Attention served as an educational implementation that demonstrated the complete neural network training pipeline without pretrained language models. Although its performance was lower than transformer-based approaches, it provided valuable insights into sequence modeling, attention mechanisms, and end-to-end representation learning.

9.3 Key Findings

- Classical machine learning provides a strong and computationally efficient baseline.
- Transformer architectures significantly improve contextual understanding compared to lexical methods.
- ELECTRA Base achieved the best overall predictive performance among the evaluated models.
- Retrieval-Augmented Generation enhances prediction by incorporating semantically relevant external knowledge.
- Developing a model from scratch offers a deeper understanding of neural network design and sequence learning, despite lower performance than pretrained transformers.

10. Challenges Encountered

Throughout the project, several practical and technical challenges were encountered during model development and experimentation.

- Model Selection: Identifying the most suitable architecture required evaluating multiple classical machine learning methods, transformer models, retrieval-based approaches, and a custom deep learning model.
- Hyperparameter Optimization: Learning rates, batch sizes, sequence lengths, and optimizer settings significantly influenced model performance and required extensive experimentation.
- Computational Constraints: Training transformer models demanded GPU resources and careful memory management, particularly for larger architectures.
- Retrieval Quality: The effectiveness of the RAG pipeline depended heavily on retrieving relevant contextual documents and accurately reranking them before prediction.
- Scratch Model Development: Designing and training a neural network from scratch required implementing tokenization, vocabulary construction, embedding layers, attention mechanisms, and optimization procedures without relying on pretrained language models.

These challenges provided valuable practical experience in model development, experimentation, and performance optimization for Natural Language Processing applications.

11. Conclusion

This project presented a comprehensive study of multiple approaches for solving the Smart MCQ Solver challenge. Beginning with classical machine learning techniques and progressing through transformer-based architectures, retrieval-augmented generation, and a custom BiLSTM with attention mechanism, the project explored the strengths and limitations of different Natural Language Processing paradigms.

Among the evaluated approaches, ELECTRA Base achieved the highest predictive performance on the Kaggle leaderboard, demonstrating the effectiveness of pretrained contextual language models for multiple-choice question answering. The Retrieval-Augmented Generation pipeline further illustrated the benefits of incorporating external contextual knowledge through semantic retrieval and reranking. Additionally, the scratch BiLSTM implementation provided valuable insights into sequence modeling, attention mechanisms, and end-to-end neural network training without relying on pretrained representations.

Overall, the project demonstrates a complete end-to-end workflow for developing, evaluating, and comparing multiple NLP approaches for MCQA, while highlighting the importance of systematic experimentation and empirical evaluation in selecting effective models.

12. Future Work

Several improvements can be explored to further enhance the performance and robustness of the proposed system:

- Investigate recent transformer architectures such as DeBERTa-v3, ModernBERT, and Llama-based language models.
- Develop hybrid systems that combine transformer models with Retrieval-Augmented Generation for improved contextual reasoning.
- Explore ensemble learning techniques to leverage the strengths of multiple architectures.
- Expand the knowledge base used in the RAG pipeline with larger and domain-specific document collections.

- Optimize the deployed system for faster inference using model quantization and efficient deployment frameworks.

References

1. Devlin, J., et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL-HLT, 2019.
2. Clark, K., et al. *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators.* ICLR, 2020.
3. Beltagy, I., Lo, K., & Cohan, A. *SciBERT: A Pretrained Language Model for Scientific Text.* EMNLP-IJCNLP, 2019.
4. Wolf, T., et al. *Transformers: State-of-the-Art Natural Language Processing.* EMNLP, 2020.
5. Reimers, N., & Gurevych, I. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP, 2019.
6. Johnson, J., Douze, M., & Jégou, H. *Billion-scale Similarity Search with GPUs (FAISS).* IEEE Transactions on Big Data, 2019.
7. Paszke, A., et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library.* NeurIPS, 2019.
8. Pedregosa, F., et al. *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 2011.